

M05 / STUDY COMPANION

M05 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	5
How to revise	5
DE05-01 CSV properly: parse, inspect, preserve raw	5
60-second recall	5
Recall task	5
DE05-02 JSON and JSONL from zero	5
60-second recall	5
Recall task	6
DE05-03 Paths, folders and deterministic batch files	6
60-second recall	6
Recall task	6
DE05-04 pandas profiling before transformation	6
60-second recall	6
Recall task	6
DE05-05 Cleaning with pandas and rejected rows	7
60-second recall	7
Recall task	7
DE05-06 pandas merge, cardinality and aggregation	7
60-second recall	7
Recall task	7
DE05-07 Data validation: record, batch and relationship rules	7

60-second recall	7
Recall task	8
DE05-08 HTTP and REST APIs from zero	8
60-second recall	8
Recall task	8
DE05-09 Calling APIs safely with requests	8
60-second recall	8
Recall task	8
DE05-10 Pagination and authentication without leaking secrets	8
60-second recall	8
Recall task	9
DE05-11 Retries, rate limits and partial failure	9
60-second recall	9
Recall task	9
DE05-12 Python + PostgreSQL with SQLAlchemy	9
60-second recall	9
Recall task	9
DE05-13 Configuration, environment variables and secret hygiene	10
60-second recall	10
Recall task	10
DE05-14 Logging, testing and reusable pipeline code	10
60-second recall	10
Recall task	10

DE05-15 Integrated Python data-engineering pipeline	10
60-second recall	10
Recall task	11

Read this first

QUICK REVISION BOOK

Use this only after you have studied the learner book. Each lesson keeps the canonical language and core proof, plus one recall task. This is not a replacement for hands-on practice.

Coverage: 15 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to revise

Read the lesson card, close the page, explain it in your own words, then answer the recall task. If recall is weak, return to that lesson only - not the entire module.

DE05-01 | CSV properly: parse, inspect, preserve raw

60-second recall

Expected control: the module CSV has 9 data records and seven header fields. Notebook, A5 must remain one product value. A correct checksum is evidence of exact bytes, not correctness of business content. If your parser produced eight fields on that row, you used text splitting instead of CSV parsing.

Changed retry: Create a new CSV with a quoted city value Delhi, NCR and an ID 0009.

Reconstruct the core contract of CSV properly: parse, inspect, preserve raw from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Corey Schafer - Python Tutorial: CSV Module - How to Read, Parse, and Write CSV Files](#)

Recall task

Create a new CSV with a quoted city value Delhi, NCR and an ID 0009. Parse it without losing field boundaries or leading-zero identity.

DE05-02 | JSON and JSONL from zero

60-second recall

The nested JSON is one batch document containing 3 orders. The JSONL fixture is 3 independent event records. `lines=[]` is a known empty list; `customer=null` means the customer object is absent. Do not conflate document grain with order/event grain.

Changed retry: Create a three-line JSONL file where the second event has an optional nested metadata object. Reconstruct the core contract of JSON and JSONL from zero from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task

[Refresh source: Corey Schafer - Python Tutorial: Working with JSON Data using the json Module](#)

Recall task

Create a three-line JSONL file where the second event has an optional nested metadata object. Process each line and explain compatibility without forcing all events to identical shape.

DE05-03 | Paths, folders and deterministic batch files

60-second recall

A defensible route anchors paths deliberately, sorts or selects inputs by an explicit rule, fingerprints inputs and refuses silent overwrites. If changing the terminal working directory breaks the script, path resolution is not yet reproducible.

Changed retry: Use filenames whose lexical order conflicts with a `business_date` inside each file. Implement and explain the intended ordering rule.

Move on only when the changed retry Reconstruct the core contract of Paths, folders and deterministic batch files from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Corey Schafer - Python Tutorial: Pathlib - The Modern Way to Handle File Paths](#)

Recall task

Use filenames whose lexical order conflicts with a `business_date` inside each file. Implement and explain the intended ordering rule.

DE05-04 | pandas profiling before transformation

60-second recall

Profiling should describe the raw population before correction. The fresh inventory has 6 rows; one zero quantity and one nonnumeric `unit_cost` are quality risks. Reporting a cleaned 4-row profile as if it were raw hides the source condition.

Changed retry: Profile a changed file containing a blank numeric field and one duplicate key; do not clean it. Reconstruct the core contract of pandas profiling before transformation from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Profile a changed file containing a blank numeric field and one duplicate key; do not clean it. Produce only observations and suspected risks.

DE05-05 | Cleaning with pandas and rejected rows

60-second recall

Authoritative controls for the supplied module fixture: 9 raw records; 4 accepted; 5 rejected; accepted amount 5100.00; accepted IDs MO01, MO05, MO06 and MO07. Both MO02 occurrences are rejected because the batch contains two records with the same order_id and provides no timestamp/version rule that would justify choosing one as authoritative. This is a deliberate conservative policy, not a universal deduplication rule. Reconstruct the core contract of Cleaning with pandas and rejected rows from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Use the fresh inventory fixture: valid unique SKU, positive integer quantity, nonnegative decimal cost and ISO received date. Split accepted/rejected with reasons.

DE05-06 | pandas merge, cardinality and aggregation

60-second recall

On the clean customer dimension, a many-to-one left merge should preserve accepted order count and amount. The duplicate-key dimension should raise a merge-cardinality failure when validate="many_to_one" is used. Reconstruct the core contract of pandas merge, cardinality and aggregation from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Create a small product dimension with one duplicate key and prove a many-to-one merge guard fails before aggregation; then repair the dimension and reconcile.

DE05-07 | Data validation: record, batch and relationship rules

60-second recall

Record checks parse one row; batch checks include uniqueness/reconciliation; relationship checks span datasets. Pydantic can help with record models, but cannot by itself prove batch uniqueness or foreign-key existence.

Changed retry: Add a syntactically valid customer_id that does not exist in the dimension. Reconstruct the core contract of Data validation: record, batch and relationship rules from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Corey Schafer - Python Pydantic Tutorial: Complete Data Validation Course](#)

Recall task

Add a syntactically valid `customer_id` that does not exist in the dimension. Show record validation passes while relationship validation fails.

DE05-08 | HTTP and REST APIs from zero

60-second recall

`/health` is unauthenticated JSON 200. `/v1/*` endpoints require the fictional bearer token. `/bad-content` is deliberately 200 text/plain. A 200 status alone is not usable-data proof.

Changed retry: Add a local endpoint/fixture expectation where status is 200 but required field is absent. Reconstruct the core contract of HTTP and REST APIs from zero from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-

[Refresh source: DataTalksClub - Docker for Data Engineering: Postgres, Docker Compose, and Real-World Workflows](#)

Recall task

Add a local endpoint/fixture expectation where status is 200 but required field is absent. Explain why transport succeeded while contract validation failed.

DE05-09 | Calling APIs safely with requests

60-second recall

Safe sequence: timeout -> status policy/raise_for_status -> media type -> parse -> payload shape. A 404 is an HTTP client error, a connection refusal is transport failure, and 200 text/plain is a response-contract failure for a JSON endpoint.

Changed retry: Call a path returning 404 and prove your client does not misclassify it as JSON success or transient server failure.

Move on only when the changed retry works with this re Reconstruct the core contract of Calling APIs safely with requests from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute m

[Refresh source: Requests Quickstart \(official\)](#)

Recall task

Call a path returning 404 and prove your client does not misclassify it as JSON success or transient server failure.

DE05-10 | Pagination and authentication without leaking secrets

60-second recall

The simulator has 7 API orders. With page sizes 2 or 4, final fetched count and unique ID count must both equal 7 and match total. Never print/log the Authorization token.

Changed retry: Fetch the same seven records with a different page size and order of page requests under a controlled replay; prove logical completeness/uniqueness remains the same.

Move on only when the changed retry works with this review closed and you can Reconstruct the core contract of Pagination and authentication without leaking secrets from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Using Python to consume an API with pagination - Renato Coelho](#)

Recall task

Fetch the same seven records with a different page size and order of page requests under a controlled replay; prove logical completeness/uniqueness remains the same.

DE05-11 | Retries, rate limits and partial failure

60-second recall

After /reset, /v1/flaky returns one 500 then 200; /v1/rate-limit returns one 429 with Retry-After=1 then 200. A bad/missing token returns 401 and should not be blindly retried as a transient failure.

Changed retry: Change retry budget from 4 to 2 and explain what evidence would show final failure. Reconstruct the core contract of Retries, rate limits and partial failure from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Requests Quickstart \(official\)](#)

Recall task

Change retry budget from 4 to 2 and explain what evidence would show final failure. Do not create an infinite retry loop.

DE05-12 | Python + PostgreSQL with SQLAlchemy

60-second recall

Use Engine.begin() or an explicit transaction boundary for grouped writes; use text() with bound parameters. Learner PostgreSQL evidence is preferred because M03 established PostgreSQL. Reconstruct the core contract of Python + PostgreSQL with SQLAlchemy from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: SQLAlchemy Unified Tutorial - Engine and transactions \(official\)](#)

Recall task

Insert a different two-row dataset inside one transaction, deliberately violate a unique key, and prove the transaction boundary behavior.

DE05-13 | Configuration, environment variables and secret hygiene

60-second recall

Required config should fail near startup. Real secrets do not belong in source, .env.example, saved logs or screenshots. Safe operational values such as batch date/page size may be logged if policy allows.

Changed retry: Switch the input directory via environment variable without editing program logic; then remove one required variable and prove failure occurs before outputs.

Move on only when the changed retry works with thi Reconstruct the core contract of Configuration, environment variables and secret hygiene from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Switch the input directory via environment variable without editing program logic; then remove one required variable and prove failure occurs before outputs.

DE05-14 | Logging, testing and reusable pipeline code

60-second recall

Tests should assert invariants. Logs should include safe run context and counts, not secrets. Re-running the same source must not append duplicate logical rows. A second successful run with doubled output is an idempotency failure.

Changed retry: Write a test for invalid date and one for duplicate ID policy. Reconstruct the core contract of Logging, testing and reusable pipeline code from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency](#)

Recall task

Write a test for invalid date and one for duplicate ID policy. Break one rule intentionally, observe failure, repair only that rule and rerun.

DE05-15 | Integrated Python data-engineering pipeline

60-second recall

Do not evaluate only the final CSV. Check exact source identity, raw/accepted/rejected reconciliation, accepted-key uniqueness, every rejection reason, accepted amount, failure behavior and rerun behavior. Then close this review before the fresh inventory retest.

Changed retry: After reviewing your weak points, close all review material and solve M05_Fresh_Checkpoint_TODO.py from the fresh inventory fixture. Reconstruct the core contract of Integrated Python data-engineering pipeline from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep.

[Refresh source: DataTalksClub - Docker for Data Engineering: Postgres, Docker Compose, and Real-World Workflows](#)

Recall task

After reviewing your weak points, close all review material and solve M05_Fresh_Checkpoint_TODO.py from the fresh inventory fixture. Keep it separate from the first attempt.